



Using Word Embedding to Enable Semantic Queries in Relational Databases

Rajesh Bordawekar
IBM T. J. Watson Research Center
Yorktown Heights, NY 10598
bordaw@us.ibm.com

Oded Shmueli*
Computer Science Department, Technion
Haifa 32000, Israel
oshmu@cs.technion.ac.il

ABSTRACT

We investigate opportunities for exploiting Artificial Intelligence (AI) techniques for enhancing capabilities of relational databases. In particular, we explore applications of Natural Language Processing (NLP) techniques to endow relational databases with capabilities that were very hard to realize in practice. We apply an unsupervised neural-network based NLP idea, Distributed Representation via Word Embedding, to extract latent information from a relational table. The word embedding model is based on meaningful textual view of a relational database and captures inter-/intra-attribute relationships between database tokens. For each database token, the model includes a vector that encodes these contextual semantic relationships. These vectors enable processing a new class of SQL-based business intelligence queries called cognitive intelligence (CI) queries that use the generated vectors to analyze contextual semantic relationships between database tokens. The cognitive capabilities enable complex queries such as semantic matching, reasoning queries such as analogies, predictive queries using entities not present in a database, and using knowledge from external sources.

CCS CONCEPTS

• **Information systems** → **Online analytical processing engines; Relational database query languages; Data analytics; Data warehouses;**

ACM Reference format:

Rajesh Bordawekar and Oded Shmueli. 2017. Using Word Embedding to Enable Semantic Queries in Relational Databases. In *Proceedings of DEEM'17, Chicago, IL, USA, May 14, 2017*, 4 pages.
DOI: <http://dx.doi.org/10.1145/3076246.3076251>

1 INTRODUCTION

We begin with a simple observation: there is a large amount of untapped latent information within a database relation. This is intuitively clear for columns that contain unstructured text. But even columns that contain different types of data, e.g., strings, numerical values, images, dates, etc., possess significant latent information in

the form of inter- and intra-column relationships. The usual way to utilize this information is using SQL and extensions, such as text extensions, or User Defined Functions (UDFs) to handle exotic data types. However, these extensions are rather limited in their *smarts*. Specifically, SQL queries rely on *value-based* analytics to detect patterns. In addition, the relational data model neglects many inter- or intra-column relationships. Thus, the traditional SQL queries lack a *holistic* view of the underlying relations, and thus are unable to extract and exploit semantic relationships that are *collectively* generated by tokens in a database relation.

In this paper, we introduce and explore a new class of business intelligence queries, called *Cognitive Intelligence (CI)* queries, that extract information from a database based, in part, on the relationships among database tokens encoded as vectors produced by a machine learning method. We use an idea from the Natural Language Processing domain, Distributed Representation via Word Embedding, to associate a vector to each database-associated token, and these vectors are then used during the query execution. We first discuss various aspects of the word embedding approach and illustrate novel capabilities of the CI queries such as semantic similarity queries, predictive queries using tokens not present in a database, and analogy queries. Finally, we enumerate key performance issues and outline future plans.

2 USING THE WORD EMBEDDING MODEL

The idea of word embedding is to fix a d -dimensional vector space and for each word in a text corpus (i.e., collection of documents), associate a dimension d vector of real numbers that encodes the *meaning* of that word [2]. Meaning of a word is determined by other words in its neighborhood. Thus, for a given text corpus, the final meaning of a word reflects collective contributions of neighborhood words for different appearances of the word in the corpus. Two words are *closely related* or *have similar meaning* if they appear often within close proximity of the same words. If two words have similar meaning, their word vectors point in very similar directions, i.e., the cosine similarity between their vectors is high (vector similarity is measured as a cosine of the angle between two vectors and can vary from 1.0 to -1.0).

Over the last few decades, a number of methods have been introduced for computing vector representations of words in a natural language [2]. Recently, a neural network based approach, word2vec [7, 8], has gained prominence as the vectors it produces appear to capture syntactic as well semantic properties of words. There are alternative mechanisms for producing vectors of similar quality, for example GloVe [9] (we use word2vec as the method for constructing vectors from database tokens, although, we could use GloVe as well). The vectors produced by word2vec seem to capture

*Work done while the author was visiting the IBM T. J. Watson Research Center.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DEEM'17, Chicago, IL, USA

© 2017 ACM. 978-1-4503-5026-6/17/05...\$15.00

DOI: <http://dx.doi.org/10.1145/3076246.3076251>

closeness of words and have been tested convincingly in a number of benchmark tests. These vectors seem to capture syntactic (e.g., present-past, singular-plural) as well as semantic closeness of words. One surprising application of word-embedding vectors is their usage in solving inductive reasoning problems such as computing analogies [10, 11], e.g., *king to man is what to woman?* (answer: queen) by using vector algebra calculations [6].

In the database context, vectors may be produced by either learning on the database itself or using external text sources. For learning from a database, a natural way of generating vectors is to apply the word embedding method to a token sequence generated from the database: each row would correspond to a sentence and a relation would correspond to a document. Thus, vectors enable a *dual view* of the data: relational and meaningful text. Word embedding then can extract latent semantic information in terms of word (and, in general, token) associations and co-occurrences and encode it in word vectors. Thus, the vectors capture first inter- and intra-attribute relationships within a row (sentence) and then aggregate these relationships across the relation (document) to compute the *collective semantic* relationships. The encoded semantic information can be then used in querying the database.

In this work, we propose to build a cognitive database system by integrating word embedding techniques and capabilities into a traditional database system [3]. We are currently developing a Spark [1] based implementation (supporting both Python and Scala) of our cognitive database system. We are using Spark DataFrames API and exploiting Spark SQL functionalities in Spark version 2.1 for building and querying our cognitive database system. Figure 1 presents the three key phases in the end-to-end execution flow of a cognitive database. The first, optional, training phase takes place when the database is used to train the model: a database table data is first *textified* into a meaningful text format. Note that, which rows are textified can be controlled using standard relational operations (e.g., by defining a view). Next, we use a modified version of the word2vec algorithm to learn vectors for the words (database tokens) in the extracted text. This phase can also use an external source, e.g., Wikipedia, as the source for text for model training. The result is a set of low-dimensional (say, of dimension 200) vectors, each representing a word. We use *word* as a synonym to token although some tokens may not be valid words in any natural language. Following vector training, the resultant vectors are stored in a relational system table (phase 2). At runtime, the SQL query execution engine uses various UDFs that fetch the trained vectors from the system table as needed and answer CI queries (phase 3). These UDFs compute distances between vectors in a semantic vector space using the cosine distance metric to determine *contextual semantic similarities* between corresponding source database tokens. The similarity results are then used in the relational query execution, thus enabling the relational engine to exploit latent semantic information for answering enhanced relational queries.

The power of CI queries goes far beyond analytical capabilities present in current relational systems. All well-known commercial and open source (e.g., Spark or MADlib[5]) database systems have built-in analytics capabilities. However, such systems view the database as a repository of features for the analytics libraries. Systems based on statistical relational learning models combine

probabilistic graphical models and first-order logic to encode uncertain first-order logic rules based on *known* information [12]. In contrast, the cognitive database learns a neural network model that infers the *meaning* of the database tokens by extracting *latent* features from the database data. The meaning is a reflection of the context (i.e., the relational view) used for training, and is encoded as vectors in a d dimensional space (d is usually 200). These vectors are then used to perform semantic matching on database tokens, rather than syntactical (value) matching. The vector-based query execution enables the cognitive database to support *inductive reasoning* queries such as analogies. Thus, the CI queries augment the capabilities of the traditional relational BI queries and can also be used in conjunction with the existing SQL operators (e.g., ROLLUP or CUBE [4]). In summary, we believe this work is a step towards empowering database systems with built-in AI capabilities.

3 CI QUERIES IN PRACTICE

Broadly, there are two classes of cognitive intelligence queries: *similarity* and *prediction* queries. Similarity queries on a database use a model that is trained using the database being queried. The prediction queries use a model that is externally trained using an unstructured data source or another database. Both classes of queries use UDFs to compute similarity. A UDF takes as input either a pair of tokens or a pair of sets (or sequences) of tokens (either constant tokens or ones associated with query variables); it then fetches vectors that correspond to these tokens and computes and returns numeric similarity value between 1.0 and -1.0 based on these vectors. A user can then control the results of the CI query by using a numerical bound for similarity result value as a predicate for selecting the eligible rows. The similarity value captures the semantic closeness between the relational variables: higher is the similarity value, closer are the two database variables. The `similarityUDF()` computes similarities between a pair of tokens by computing the cosine distance on the corresponding vectors. For sets and sequences, the individual pair-wise similarity values are then aggregated to generate the resultant value. In case of sequences, computation of the final similarity value takes into account the ordering of tokens: different pair-wise distances contribute differently to the final value based on their relative ordering. For example, two food items, a `chicken_item` consisting of (chicken, salt), is not very similar to a `corn_item` consisting of (corn, salt), although both contain salt (however, the `corn_item` is closer to `chicken_item` than a `wheat_item` that contains (wheat, sugar)).

The first example (Figure 2) illustrates an SQL CI query that identifies similar customers by comparing their purchases. Assume that `sales` is a table that contains all customer transactions for a grocery store and whose `sales.purchase` column contains all items purchased during a transaction. The current query uses a UDF, `similarityUDF()`, that computes similarity match between two sets of vectors, that correspond to the items purchased by the corresponding customers. Unlike the food item scenario, the purchased item list can be viewed as an unordered bag of items; and individual pair-wise distances contribute equally to the final result. A modified version of the query (not shown) can identify similar customers based on their overall purchasing pattern as evidenced in a number of rows. The word embedding model creates

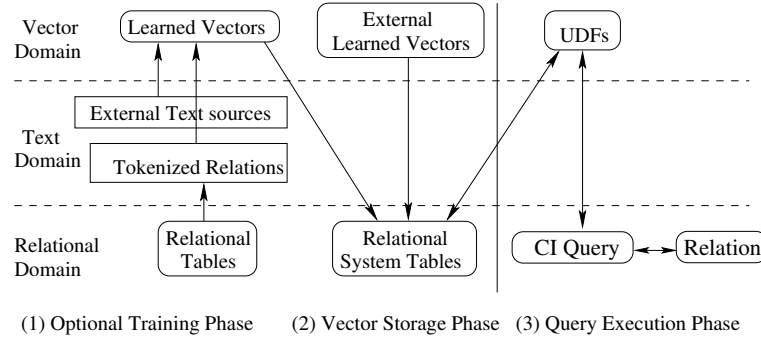


Figure 1: End-to-end Execution Flow of a Cognitive Relational Database

```
SELECT X.custID, X.name, Y.custID, Y.name,
similarityUDF(X.purchase, Y.purchase) AS similarity
FROM sales X, sales Y
similarityUDF(X.purchase, Y.purchase) > 0.5
ORDER BY X.name, similarity
LIMIT 10
```

Figure 2: Example of an CI similarity query: find similar customers based on their purchased items

```
SELECT P1.type, P2.type
FROM Products P1, Products P2
WHERE analogyUDF(P1.type, 'peanut-butter', 'jelly', P2.type)
= (SELECT MAX (analogyUDF(P3.type, 'peanut-butter',
'jelly', P2.type)))
FROM Products P3
```

Figure 3: Example of an CI analogy query

a vector for each customer name that captures the *overall* purchases made by that customer. Then, the customers with similar purchase patterns would have vectors that are close using the cosine distance metric. The pattern observed in this query can be applied to other domains as well, e.g., identifying patients that are taking similar drugs, but with different brand names or identifying food items with similar ingredients, or finding mutual funds with similar investment strategies. In all these scenarios, the relational database tokens are matched based on their *semantic* similarities, not by their values. This is a fundamental change from the current relational model which uses *value* based matching for the core relational operations (i.e., select, project, and join). In a cognitive database these relational operations can be strengthened with semantic matching.

A unique capability of word-embedding vectors is that one can use them to answer inductive reasoning questions that require an individual to reason from part to whole, or from particular to general [11]. An example of inductive reasoning is computing analogies; e.g., king to man is like queen to woman. Analogy can be viewed as a special type of similarity, where two pairs of entities that share similar relationships, have similar distance between their

```
SELECT X.number, X.name, similarityUDF(X.purchase,
'allergenic') AS similarity
FROM sales X
similarityUDF(X.purchase, 'allergenic') > 0.3
LIMIT 10
```

Figure 4: Example of a prediction query: find customers that have purchased allergic items

corresponding vectors [6, 8]. CI queries enable exposing such relationships within a relational table. For example, the following query (Figure 3) identifies pairs of products that relate to each other like tokens peanut butter and jelly (both are product types). The key is that if product p_1 relates to product p_2 like peanut butter to jelly, their associated vectors $\vec{p_1}$, $\vec{p_2}$, $\vec{peanut} \rightarrow \vec{butter}$ and $\vec{jelly}$ are likely to satisfy that the vector differences $(\vec{peanut} \rightarrow \vec{butter} - \vec{jelly})$ and $(\vec{p_1} - \vec{p_2})$ are cosine-similar (we use the UDF `analogyUDF()` to compute the differences). Potential answers to this query may include (chips, salsa) or (pancake, maple syrup). The analogy capabilities of CI queries have several applications in the enterprise space, e.g., associating customers with either most-common or least-common purchases in a given domain (e.g., books, electronics, etc.)

The final use case provides an illustration of using an external text source for querying a database (Figure 4). The example assumes that we have built a word embedding model using an external source such as Wikipedia. Let's assume that the external knowledge source has a document that lists all allergenic fruits, e.g., *list of allergenic fruits include strawberries, raspberries, blueberries,...* The model will create vectors for all these words and the vector for the word *allergenic* will be closer to the vectors of *strawberries*, *raspberries*, and *blueberries*. Now, we can import this model and use it to query the sales database to find out which customers have bought items that may be allergenic, as defined by the external source. As Figure 4 shows, the `similarityUDF()` UDF is used to identify those purchases that contain items similar to *allergenic*, such as *strawberries*. This example demonstrates a very powerful ability of CI queries that enables users to query a database using a token (e.g., *allergenic*) not present in the database. This capability can be applied to different scenarios in which recent, updatable information, can be used to query historical data. For example, a model

built using a FDA recall notices could be used to identify those customers who have purchased medicines similar to the recalled medicines.

The key characteristic of the CI queries is that these queries are executed, in part, using the vectors in the word embedding model. If the word embedding model is generated using the database being queried, it captures meaning in the context of the associated relational table, as specified by the relational view. If a model is rebuilt using a different relational view, a CI query may return different results for the new model. For every token in the database, *irrespective of its type*, the model generates a single-precision floating point vector of dimension d . In other words, similarity calculations over SQL variables of *different* types always use *uniformly* typed vectors. This enables the database system to seamlessly analyze data of different types (e.g., text, numeric values, and potentially, images) using a single CI query. We are currently working on extending our infrastructure to support additional data types such as numeric values, SQL Dates, and images.

None of the existing relational systems supports the type of queries outlined in this section. It is conceivable that one may obtain similar results by exporting data from a database table and analyzing it externally. However, our approach provides seamless integration of the word embedding model into the existing standard SQL infrastructure, including the query optimizer, thus extending capabilities of existing systems to support *smarter* SQL queries.

Although we used hypothetical scenarios to demonstrate our ideas, CI queries are applicable to a broad class of domains. These include retail, customer care, log analytics, healthcare, genomics, semantic search over documents (patent, legal, or financial), healthcare informatics, and human resource management.

3.1 Implementation Details

In our end-to-end Spark implementation, we use word2vec to train the model using either a dataset exported from a database (e.g., a CSV file) or text documents from an external corpus (e.g., wikipedia). Alternatively, we can load a pre-trained model directly. The Spark program loads the source database and the model into dataframes. We also build additional hash-based indices to accelerate vector computations. In an end-to-end system, there are three spots where performance can become a concern. First, in the training phase, if the vectors have to be trained from the raw database data, depending on the size of the text used, the vector training time can be very long. However, this training phase may not always be necessary as the system can use pre-trained vectors, and if the vectors need to be trained, the training (or retraining for new data), can be implemented as a batch process. Performance of training can be further improved by using GPUs. Second, the cost of accessing the trained vectors from the system tables has to be as small as possible as it impacts query execution time. Finally, the execution cost of a CI query is dependent on the performance of the distance function. In many cases, we may need to compute distances among a large number of vectors (e.g., for analogy queries). The distance calculations can be accelerated either using CPU's SIMD capabilities or using accelerators such as GPUs. This is a focus of our current work on accelerating key Spark ML libraries and we will provide details in subsequent publications.

4 CONCLUSIONS AND FUTURE WORK

We describe the enhanced querying capabilities resulting from having each database token associated with a vector that captures its syntactic as well as its semantic characteristics. Vectors are obtained by a learning algorithm operating on database-derived text. We use these vectors to enhance database querying capabilities. In essence, these vectors provide another way to look at the database, almost orthogonal to the structured relational regime, as vectors enable a dual view of the data: relational and meaningful text. We thereby introduce and explore a new class of queries called *cognitive intelligence* (CI) queries that extract information from the database based, in part, on the relationships encoded by these vectors.

We are implementing a prototype system on top of Apache Spark [1] to exhibit the power of CI queries. We are currently extending our infrastructure to support more complex query patterns that use different data types, e.g., numeric values and images. We are also working on accelerating model training and vector distance calculations using GPUs, and developing new techniques for incremental vector training. We believe CI queries are applicable to a broad class of application domains including healthcare, bio-informatics, document searching, retail analysis, and data integration. We are currently working on applying the CI capabilities to some of these domains.

REFERENCES

- [1] Apache Foundation. 2017. Apache Spark: A fast and general engine for large-scale data processing. (2017). Release 2.1.
- [2] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. 2003. A Neural Probabilistic Language Model. *Journal of Machine Learning Research* 3 (2003), 1137–1155. <http://www.jmlr.org/papers/v3/bengio03a.html>
- [3] Rajesh Bordawekar and Oded Shmueli. 2016. Enabling Cognitive Intelligence Queries in Relational Databases using Low-dimensional Word Embeddings. *CoRR abs/1603.07185* (2016). <http://arxiv.org/abs/1603.07185>
- [4] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. 1997. Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals. *Data Mining and Knowledge Discovery* 1, 1 (1997), 29–53.
- [5] Joseph M. Hellerstein, Christopher Re, Florian Schoppmann, Daisy Zhe Wang, Eugene Fratkin, Aleksander Gorajek, Kee Siong Ng, Caleb Welton, Xixuan Feng, Kun Li, and Arun Kumar. 2012. The MADlib analytics library: or MAD skills, the SQL. *Proc. VLDB Endow.* 5, 12 (August 2012).
- [6] Omer Levy and Yoav Goldberg. 2014. Linguistic regularities in sparse and explicit word representations. In *Eighteenth Conference on Computational Natural Language Learning*. Association for Computational Linguistics.
- [7] Tomas Mikolov. 2013. word2vec: Tool for computing continuous distributed representations of words. (2013). github.com/tmikolov/word2vec.
- [8] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and their Compositionality. In *27th Annual Conference on Neural Information Processing Systems 2013*. 3111–3119. <http://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>
- [9] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. 1532–1543. <http://aclweb.org/anthology/D/D14/D14-1162.pdf>
- [10] D. E. Rumelhart and A. A. Abrahamson. 1973. A model for analogical reasoning. *Cognitive Psychology* 5, 1 (1973).
- [11] R. J. Sternberg and M. K. Gardner. 1983. Unities in inductive reasoning. *Journal of Experimental Psychology: General* 112, 1 (1983).
- [12] Daisy Zhe Wang, Yang Chen, Christan Grant, and Kun Li. 2014. Efficient In-Database Analytics with Graphical Models. *IEEE Data Engineering Bulletin* (2014).